IBM Developer
SKILLS NETWORK

Winning Space Race with Data Science

AKRITI THAKUR

Outline

- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

Introduction

- we will predict if the Falcon 9 first stage will land successfully. SpaceX advertises Falcon 9 rocket launches on its website, with a cost of 62 million dollars; other providers cost upward of 165 million dollars each, much of the savings is because SpaceX can reuse the first stage. Therefore if we can determine if the first stage will land, we can determine the cost of a launch. This information can be used if an alternate company wants to bid against SpaceX for a rocket launch. tasked with predicting if the first stage of the SpaceX Falcon 9 rocket will land successfully. With the help of your Data Science findings and models, the competing startup you have been hired by can make more informed bids against SpaceX for a rocket launch

Section 1

Methodology

Methodology

Executive Summary

- Data collection methodology:

✓**By fetching data from Api.**

✓**By web scrapping using beautiful soup.**

- Perform exploratory data analysis (EDA) using visualization and SQL
- Perform interactive visual analytics using Folium and Plotly Dash
- Perform predictive analysis using classification models

Data Collection

Collecting and working with data in pandas is straightforward and powerful. Here's a basic guide to get you started:

1. Importing pandas

First, you need to import the pandas library. If you haven't installed it yet, you can do so using pip:

pip install pandas

Then, import pandas in your Python script or Jupyter notebook:

import pandas as pd

AI-generated code. Review and use carefully

2. Loading Data

You can load data from various sources such as CSV files, Excel files, SQL databases, and more. Here are some common methods:

•**From a CSV file:**

df = pd.read_csv('file_path.csv')

AI-generated code. Review and use careful

•**From an Excel file:**

df = pd.read_excel('file_path.xlsx')

AI-generated code. Review and use carefully.

•**From a SQL database:**

import sqlite3 conn = sqlite3.connect('database.db') df = pd.read_sql_query("SELECT * FROM table_name", conn)

AI-generated code. Review and use carefully

3. Exploring Data

Once you have loaded the data into a DataFrame, you can explore it using various pandas functions:

•**View the first few rows:**

print(df.head())

AI-generated code. Review and use carefully.

•**Get a summary of the DataFrame:**

print(df.info())

AI-generated code. Review and use carefully.

•**Get basic statistics:**

print(df.describe())

Data Collection – SpaceX API

Steps Explained:
1.Define the API Endpoint: Specify the URL of the API you want to fetch data from.
2.Make a GET Request: Use the fetch function to make a GET request to the specified URL.
3.Handle the Response: Check if the response is okay. If it is, convert it to JSON.
4.Process the Data: Use the retrieved data as needed.
5.Handle Errors: Catch and handle any errors that occur during the fetch process.

- https://github.com/akriti-thakur/jupyter-notebook.git
- Go to space_X folder

fetching data from api

```python
sta = "https://api.spacexdata.com/v4/rockets/"
# sta_url = "https://en.wikipedia.org/w/index.php?title=List_of_Falcon_9_and_Falcon_Heavy_launches&oldid=1027686922"
response=requests.get(sta)
response.content
if response.status_code = 200:
    data = response.json()
    df = pd.json_normalize(data)
    print(df)
else:
    print(f"Failed to retrieve data: {response.status_code}")
```

[3]

Data Collection - Scraping

- Web scraping is a powerful technique for extracting data from websites. Here's a basic guide to get you started with web scraping using Python
- https://github.com/akriti-thakur/jupyter-notebook.git
- Go to space_X folder

Webscraping using pandas

```
14]
url = "https://en.wikipedia.org/w/index.php?title=List_of_Falcon_9_and_Falcon_Heavy_launches&oldid=1027686922"
```

```
15]
try:
    df = pd.read_html(url)
except Exception as e:
    print(f"An error occurred: {e}")
```

```
response= requests.get(url)
```

```
16]
soup= BeautifulSoup(response.content,"html.parser")
```

```
17]
title = soup.title
print(f"Title of the page: {title}")
```

```
Title of the page: None
```

Data Wrangling

- Data wrangling, also known as data munging, is the process of transforming and mapping raw data into a more usable format. This process is crucial in data analysis and involves several steps to clean, organize, and prepare data for analysis. Here are the key steps involved in data wrangling:

1. Data Collection

- Gathering data from various sources such as databases, APIs, web scraping, or files.

2. Data Cleaning

- Removing or correcting inaccuracies, handling missing values, and filtering out irrelevant data. This step ensures the data is accurate and consistent.

3. Data Transformation

- Converting data into a suitable format or structure. This may involve normalizing data, aggregating data, or converting data types.

4. Data Enrichment

- Enhancing the dataset by adding additional information or context. This could involve merging datasets or adding calculated fields.

5. Data Validation

- Ensuring the data meets the required quality standards and is ready for analysis. This step involves checking for consistency, accuracy, and completeness.

6. Data Storage

- Saving the cleaned and transformed data in a database or file for further analysis.

EDA with Data Visualization

- Exploratory Data Analysis (EDA) with data visualization is a crucial step in understanding your dataset and uncovering patterns, trends, and relationships. Here’s we have done EDA using data visualization in Python, particularly with pandas and matplotlib.
- the main motive of using data visualization techniques are it will provide us an insight about various factors affecting our research.

EDA with SQL

- Performing Exploratory Data Analysis (EDA) with SQL in a Jupyter Notebook is a powerful way to combine the strengths of SQL for data retrieval and Python for data analysis and visualization.

- https://github.com/akriti-thakur/jupyter-notebook.git
- Go to space_X folder

Build an Interactive Map with Folium

- Summarize what map objects such as markers, circles, lines, etc. you created and added to a folium map
- Explain why you added those objects
- Add the GitHub URL of your completed interactive map with Folium map, as an external reference and peer-review purpose

Build a Dashboard with Plotly Dash

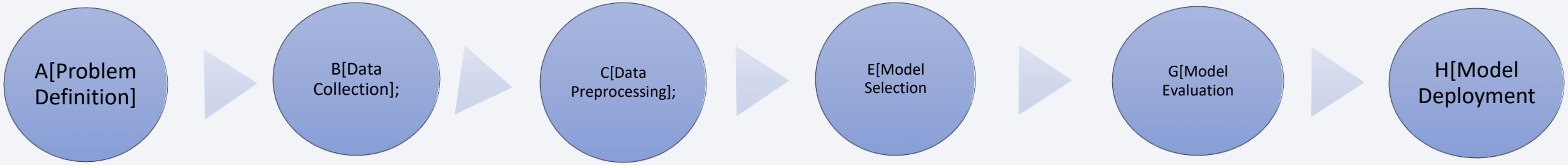
- **Plots and Graphs**

1. **Line Charts**:
    1. **Usage**: To show trends over time.
    2. **Example**: Sales over the past year.
2. **Bar Charts**:
    1. **Usage**: To compare quantities across different categories.
    2. **Example**: Sales by region.
3. **Pie Charts**:
    1. **Usage**: To show proportions of a whole.
    2. **Example**: Market share of different products.
4. **Histograms**:
    1. **Usage**: To show the distribution of a single variable.
    2. **Example**: Distribution of customer ages.
5. **Scatter Plots**:
    1. **Usage**: To show relationships between two variables.
    2. **Example**: Sales vs. advertising spend.
6. **Heatmaps**:
    1. **Usage**: To show data density or relationships between variables.
    2. **Example**: Correlation matrix of different features.

- Plots and graph make our report more eye-catching and provide use more useful insights.

- https://github.com/akriti-thakur/jupyter-notebook.git
- Go to space_X folder

Predictive Analysis (Classification)

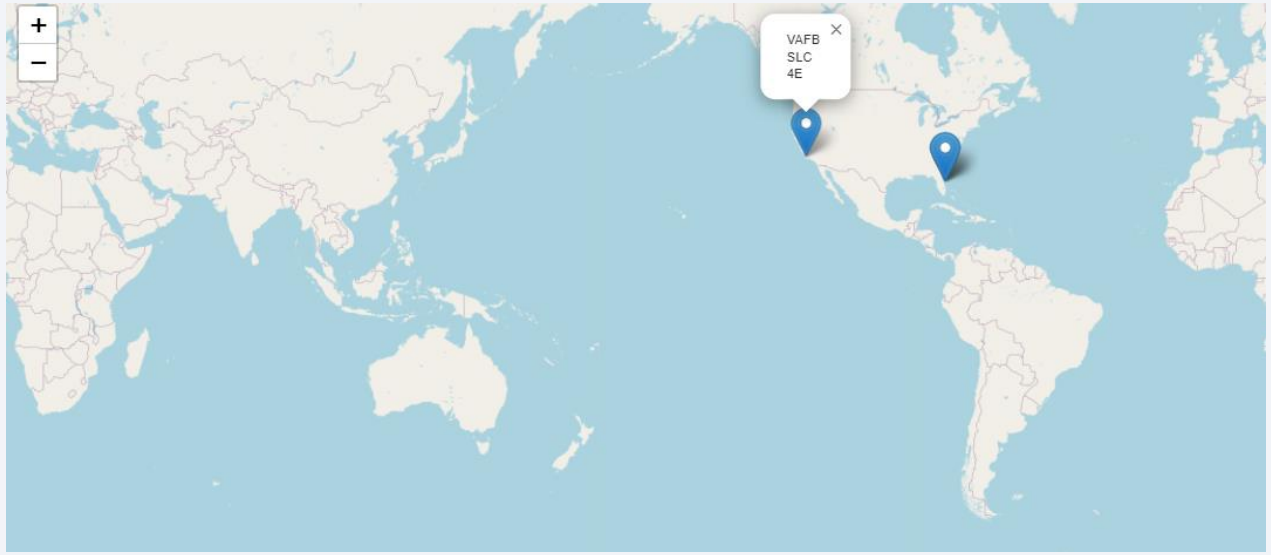


•**Model Maintenance**: Continuously monitor the model's performance, update it with new data, and retrain as needed.

- https://github.com/akriti-thakur/jupyter-notebook.git
- Go to space_X folder

Results

Interactive analytics demo in screenshots



Predictive analysis results

Find the method performs best:

```
In [32]:
print('Accuracy for Logistics Regression method:', logreg_cv.score(X_test, Y_test))
print( 'Accuracy for Support Vector Machine method:', svm_cv.score(X_test, Y_test))
print('Accuracy for Decision tree method:', tree_cv.score(X_test, Y_test))
print('Accuracy for K nearsdt neighbors method:', knn_cv.score(X_test, Y_test))

Accuracy for Logistics Regression method: 0.8333333333333334
Accuracy for Support Vector Machine method: 0.8333333333333334
Accuracy for Decision tree method: 0.6111111111111112
Accuracy for K nearsdt neighbors method: 0.8333333333333334
```

Exploratory data analysis results

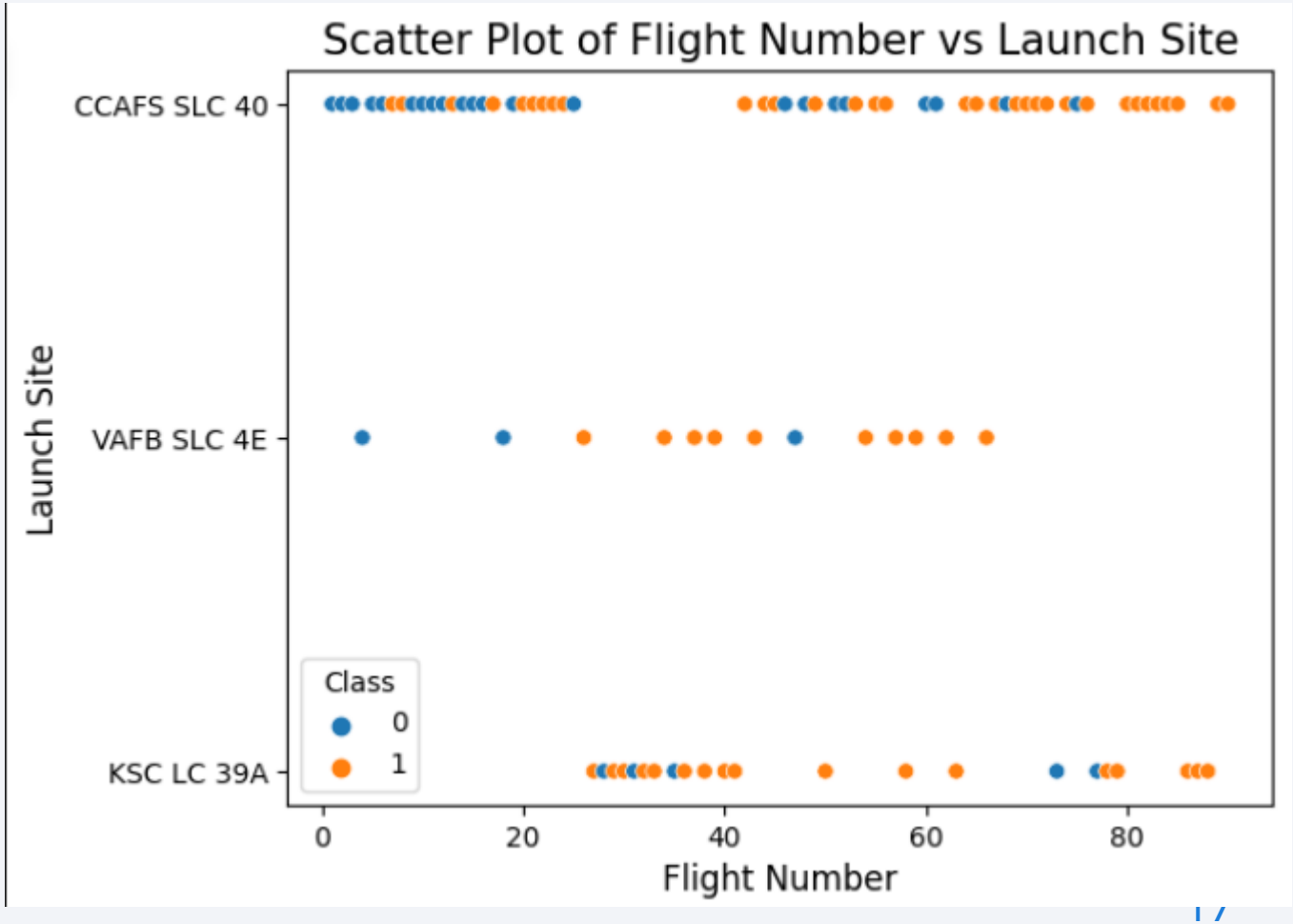
```
d=dm['LaunchSite'].value_counts()
d

# How many launches came from CCAFS SLC 40?
36]  ✓ 0.0s

LaunchSite
CCAFS SLC 40    55
KSC LC 39A      22
VAFB SLC 4E     13
Name: count, dtype: int64

success_rate = dm['Class'].mean()
print(f"sucess rate : {success_rate *100:.2f}%")
37]  ✓ 0.0s

sucess rate : 66.67%

d=dm['Orbit'].value_counts()
d
38]  ✓ 0.0s

Orbit
GTO      27
ISS      21
VLEO     14
PO        9
LEO       7
```

Section 2

Insights drawn from EDA

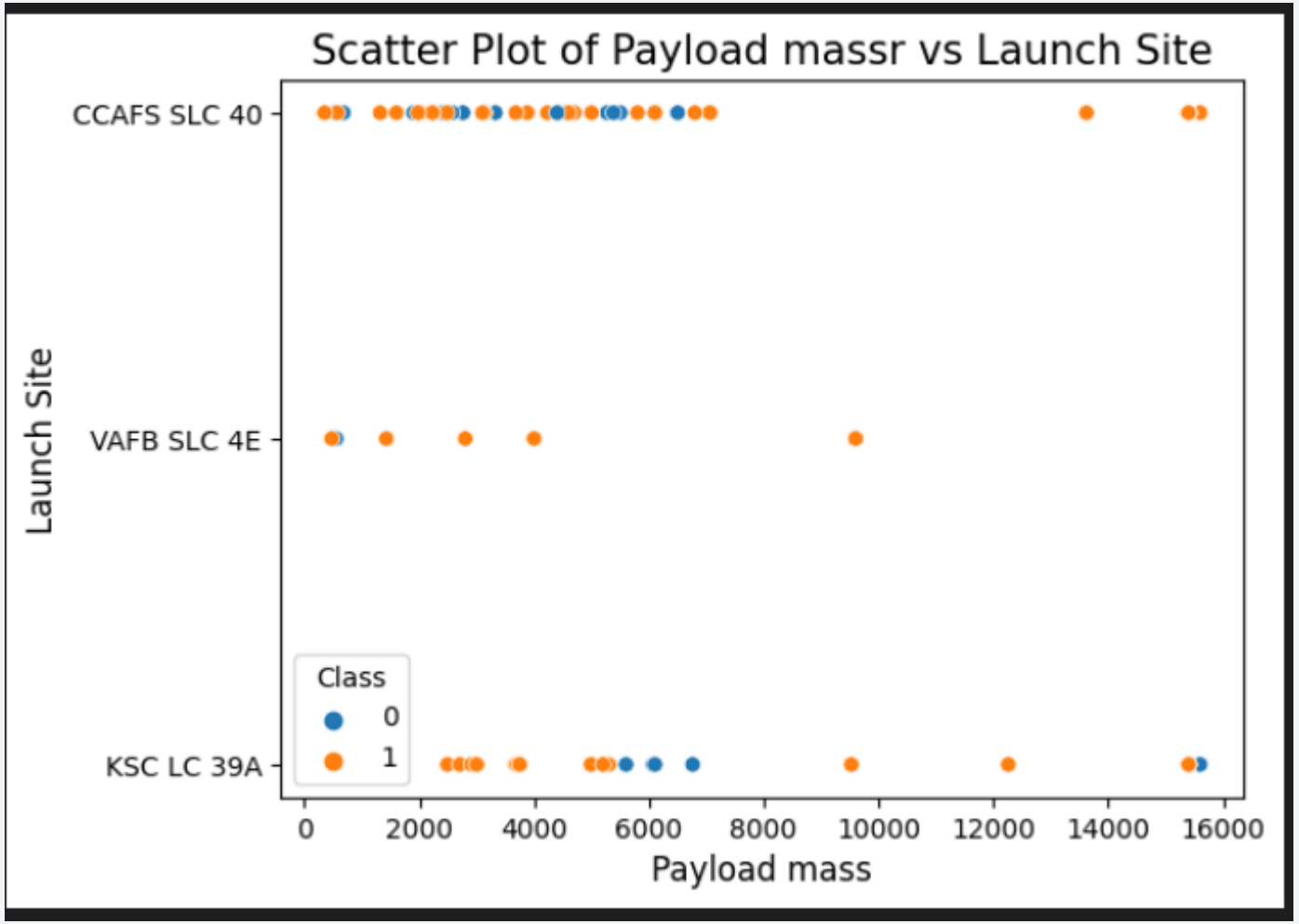
Flight Number vs. Launch Site

- This scatter plot provides a visual representation of the relationship between flight numbers and launch sites, highlighting the success and failure rates over time. It can be useful for identifying patterns and trends in launch site performance and flight outcomes



Payload vs. Launch Site

- This scatter plot provides a visual representation of the relationship between payload mass and launch site, highlighting the success and failure rates for different payload masses. It can be useful for analyzing trends in launch performance and optimizing launch strategies based on payload mass and launch site.

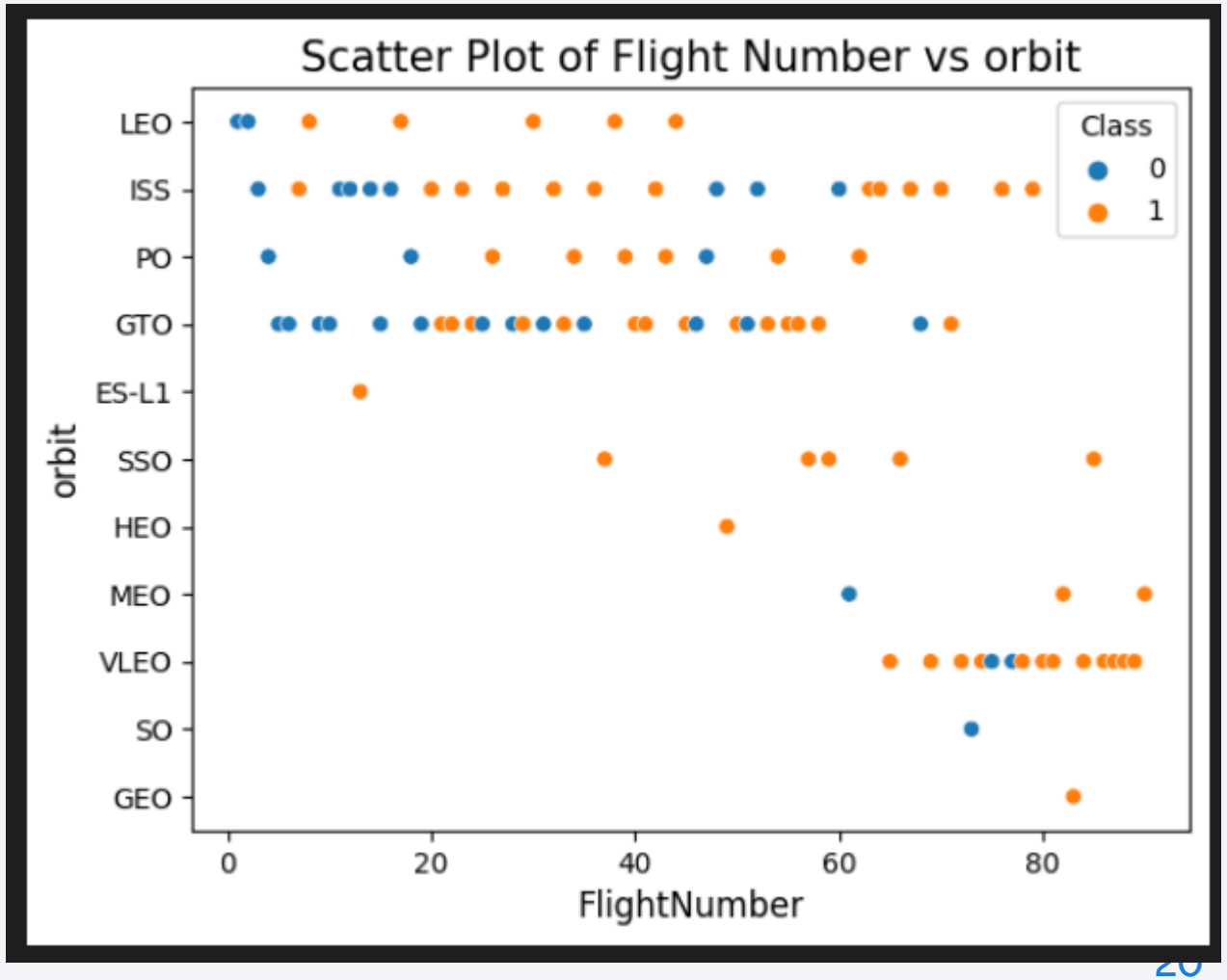


Success Rate vs. Orbit Type

This bar plot provides a visual representation of the relationship between payload mass and orbital classifications, highlighting the success and failure rates for different orbits. It can be useful for analyzing trends in mission performance and optimizing launch strategies based on the type of orbit.

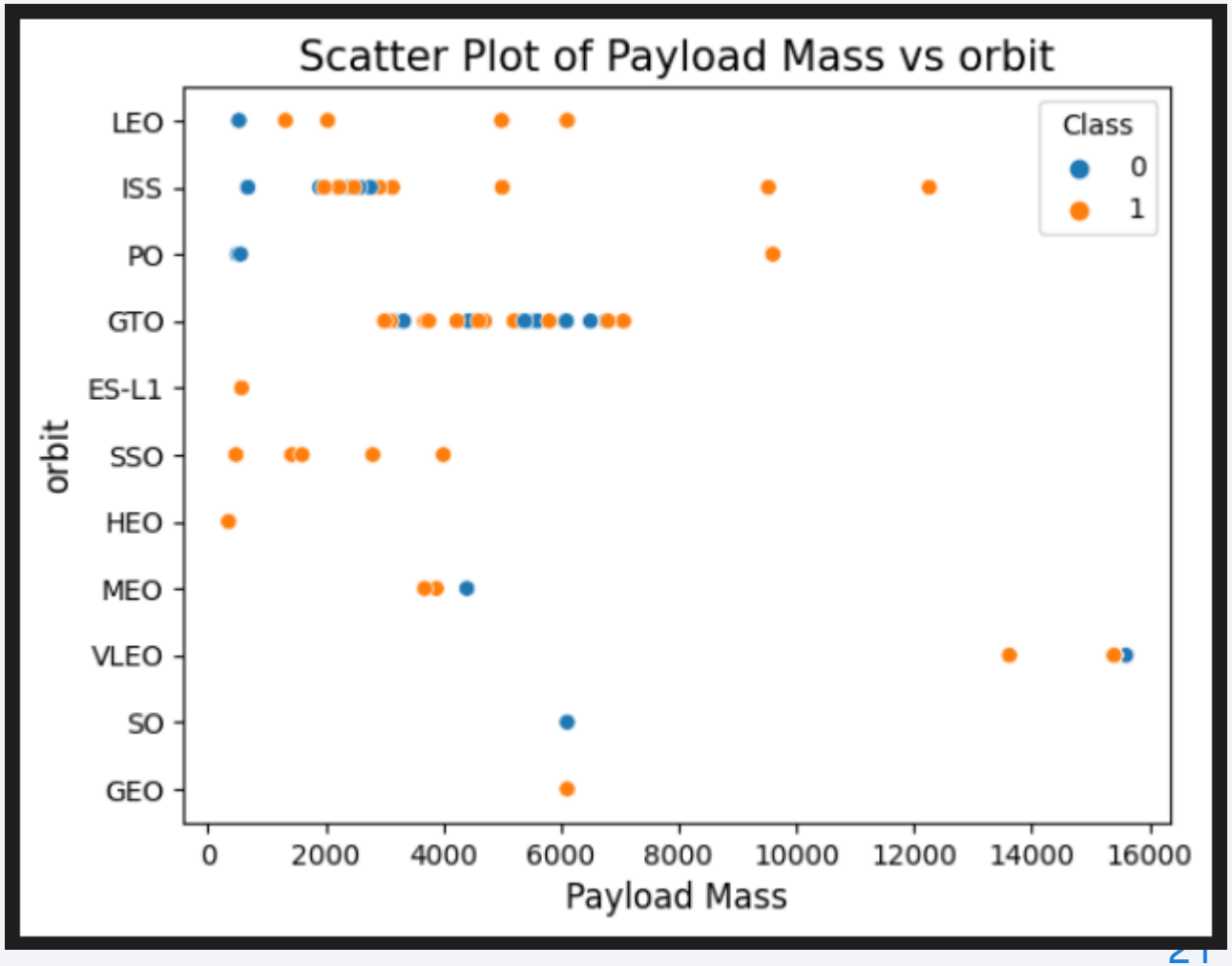
Flight Number vs. Orbit Type

- This scatter plot provides a visual representation of the relationship between flight numbers and orbital classifications, highlighting the success and failure rates for different orbits. It can be useful for analyzing trends in mission performance and optimizing launch strategies based on the type of orbit.

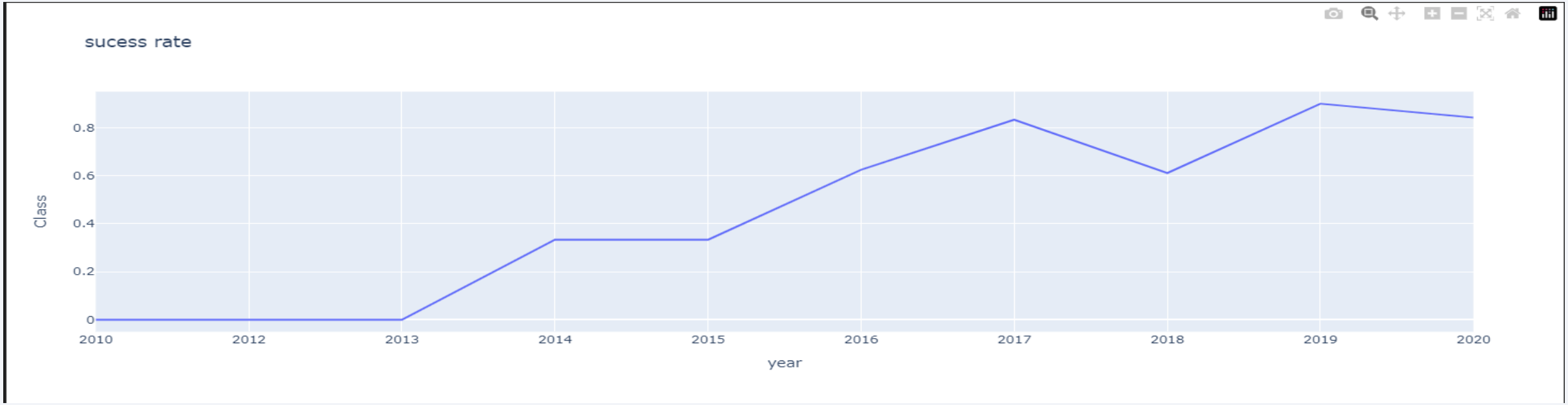


Payload vs. Orbit Type

- This scatter plot provides a visual representation of the relationship between flight numbers and orbital classifications, highlighting the success and failure rates for different orbits. It can be useful for analyzing trends in mission performance and optimizing launch strategies based on the type of orbit.



Launch Success Yearly Trend



This line graph provides a visual representation of the success rate over a decade, highlighting the overall improvement with some fluctuations. It can be useful for analyzing trends and identifying periods of significant change in success rates.

All Launch Site Names

•dm['LaunchSite'].value_counts(): This line of code counts the occurrences of each unique value in the ‘LaunchSite’ column of the DataFrame dm.

•d: The result is stored in the variable d, which is a pandas Series containing the counts of each launch site.

```python
d=dm['LaunchSite'].value_counts()
d


# How many launches came from CCAFS SLC 40?
```

✓ 0.0s Python

```
LaunchSite
CCAFS SLC 40    55
KSC LC 39A      22
VAFB SLC 4E     13
Name: count, dtype: int64
```

Launch Site Names Begin with 'CCA'

Explanation of the Query
1.SELECT *: This part of the query selects all columns from the table.
2.FROM spacexdata: Specifies the table from which to retrieve the data.
3.WHERE launch_site LIKE ‘CCA%’: Filters the records to include only those where the launch_site column starts with ‘CCA’.
4.LIMIT 5: Limits the result set to 5 records.

This query and its output provide a snapshot of SpaceX launches from the CCAFS LC-40 launch site, including details about the date, booster version, payload, orbit, customer, and mission outcomes. This information can be useful for analyzing launch patterns and performance

Display 5 records where launch sites begin with the string 'CCA'

```
%sql select* from  SPACEXTBL where "launch_site" like 'CCA%' limit 5;
```

[8] ✓ 0.0s

```
 * sqlite:///my_data1.db
Done.
```

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS__KG_	Orbit	Customer	Mission_Outcome	Landing_Outcome
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute)
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No attempt
2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No attempt
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No attempt

Total Payload Mass

This SQL query and its output provide a quick way to calculate the total payload mass for a specific customer, in this case, NASA (CRS). By using the SUM function and filtering with the WHERE clause, you can easily aggregate data based on specific criteria.

Task 3

Display the total payload mass carried by boosters launched by NASA (CRS)

```
[21]
%sql SELECT SUM("PAYLOAD_MASS__KG_") AS Total_Payload_Mass from  SPACEXTBL Where "CUSTOMER" = 'NASA (CRS)';
```

```
 * sqlite:///my_data1.db
Done.
```

Total_Payload_Mass
45596

Average Payload Mass by F9 v1.1

This SQL query and its output provide valuable insights into the average payload mass carried by SpaceX's F9 v1.1 booster version. By understanding this metric, stakeholders can make informed decisions about mission planning, budgeting, and performance evaluation.

Task 4

Display average payload mass carried by booster version F9 v1.1

```
%sql SELECT avg("PAYLOAD_MASS__KG_") AS Total_Payload_Mass from  SPACEXTBL Where "Booster_Version" = 'F9 v1.1';
```

```
 * sqlite:///my_data1.db
Done.
```

Total_Payload_Mass
2928.4

First Successful Ground Landing Date

The first successful landing on a ground pad by SpaceX occurred on December 22, 2015

Task 5

List the date when the first succesful landing outcome in ground pad was acheived.

Hint:Use min function

```
%sql SELECT MIN("Date") AS First_Successful_Landing from SPACEXTBL where "Landing_Outcome" = 'Success (ground pad)';
```

[23]

```
 * sqlite:///my_data1.db
Done.
```

First_Successful_Landing
2015-12-22

Successful Drone Ship Landing with Payload between 4000 and 6000

The first successful landing on a ground pad by SpaceX occurred on December 22, 2015. This milestone was achieved using the Falcon 9 rocket, marking a significant advancement in reusable rocket technology and space exploration.

Task 6

List the names of the boosters which have success in drone ship and have payload mass greater than 4000 but less than 6000

```
%sql SELECT DISTINCT "BOOSTER_VERSION" from  SPACEXTBL WHERE "Landing_Outcome" = 'Success (drone ship)' AND "PAYLOAD_MASS__KG_" > 4000 AND "PAYLOAD_MASS__KG_" < 6000;
```

[27]

```
 * sqlite:///my_data1.db
Done.
```

Booster_Version
F9 FT B1022
F9 FT B1026
F9 FT B1021.2
F9 FT B1031.2

Total Number of Successful and Failure Mission Outcomes

The SQL query identifies the first successful landing on a ground pad by SpaceX, which occurred on December 22, 2015. This milestone marked a significant achievement in reusable rocket technology, demonstrating SpaceX's capability to land rockets safely and efficiently, paving the way for more sustainable space missions.

Task 7

List the total number of successful and failure mission outcomes

```
[28] %sql SELECT "MISSION_OUTCOME", COUNT(*) AS Total_Count FROM SPACEXTBL GROUP BY "MISSION_OUTCOME";
```

```
 * sqlite:///my_data1.db
Done.
```

Mission_Outcome	Total_Count
Failure (in flight)	1
Success	98
Success	1
Success (payload status unclear)	1

Boosters Carried Maximum Payload

The SQL query identifies the booster versions that have carried the maximum payload mass. The query results show multiple booster versions, such as F9 B5 B1048.4 and F9 B5 B1049.4, each carrying significant payloads, demonstrating SpaceX's capability to handle heavy payloads efficiently.

Task 8

List the names of the booster_versions which have carried the maximum payload mass. Use a subquery

```
%sql SELECT "BOOSTER_VERSION" FROM SPACEXTBL WHERE "PAYLOAD_MASS__KG_" = (SELECT MAX("PAYLOAD_MASS__KG_") FROM SPACEXTBL);
```

```
 * sqlite:///my_data1.db
Done.
```

Booster_Version
F9 B5 B1048.4
F9 B5 B1049.4
F9 B5 B1051.3
F9 B5 B1056.4
F9 B5 B1048.5
F9 B5 B1051.4
F9 B5 B1049.5
F9 B5 B1060.2
F9 B5 B1058.3
F9 B5 B1051.6
F9 B5 B1060.3
F9 B5 B1049.7

2015 Launch Records

The SQL query identifies the booster versions that have carried the maximum payload mass. The results show multiple booster versions, such as F9 B5 B1048.4 and F9 B5 B1049.4, each handling significant payloads, highlighting SpaceX's capability to manage heavy payloads efficiently.

Task 9

List the records which will display the month names, failure landing_outcomes in drone ship ,booster versions, launch_site for the months in year 2015.

```
%sql SELECT substr("Date", 6, 2) AS Month, "Landing_Outcome", "BOOSTER_VERSION", "Launch_Site" FROM SPACEXTBL WHERE "Landing_Outcome" = 'Failure (drone ship)' AND substr("Date", 1, 4) = '2015';
```

[31]

```
 * sqlite:///my_data1.db
Done.
```

Month	Landing_Outcome	Booster_Version	Launch_Site
01	Failure (drone ship)	F9 v1.1 B1012	CCAFS LC-40
04	Failure (drone ship)	F9 v1.1 B1015	CCAFS LC-40

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

The SQL query identifies the booster versions that have carried the maximum payload mass. The results show multiple booster versions, such as F9 B5 B1048.4 and F9 B5 B1049.4, each handling significant payloads, highlighting SpaceX's capability to manage heavy payloads efficiently.

```
%sql SELECT "Landing_Outcome", COUNT(*) AS Outcome_Count FROM SPACEXTBL WHERE "Date" BETWEEN '2010-06-04' AND '2017-03-20' GROUP BY "Landing_Outcome" ORDER BY Outcome_Count DESC;
```

[34]

```
 * sqlite:///my_data1.db
Done.
```

Landing_Outcome	Outcome_Count
No attempt	10
Success (drone ship)	5
Failure (drone ship)	5
Success (ground pad)	3
Controlled (ocean)	3
Uncontrolled (ocean)	2
Failure (parachute)	2
Precluded (drone ship)	1

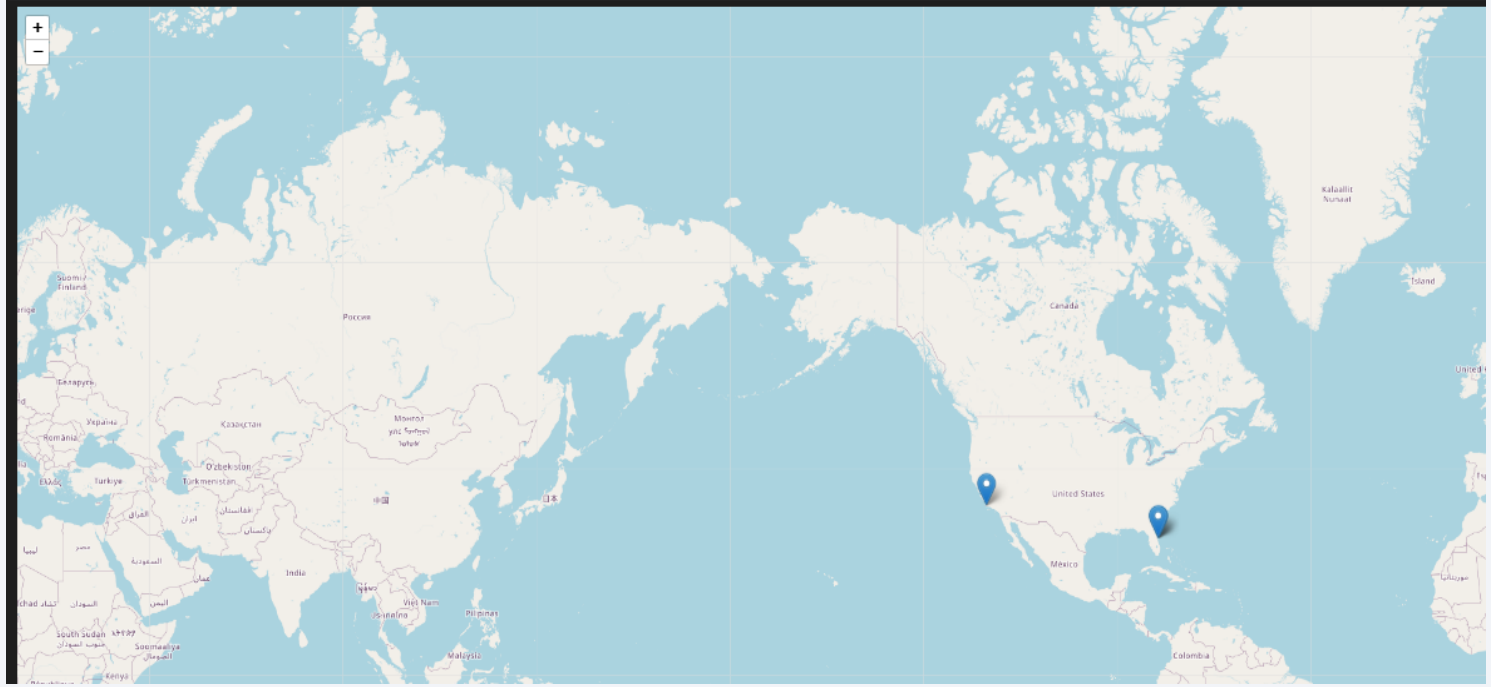
Section 3

Launch Sites Proximities Analysis

<Folium Map Screenshot 1>

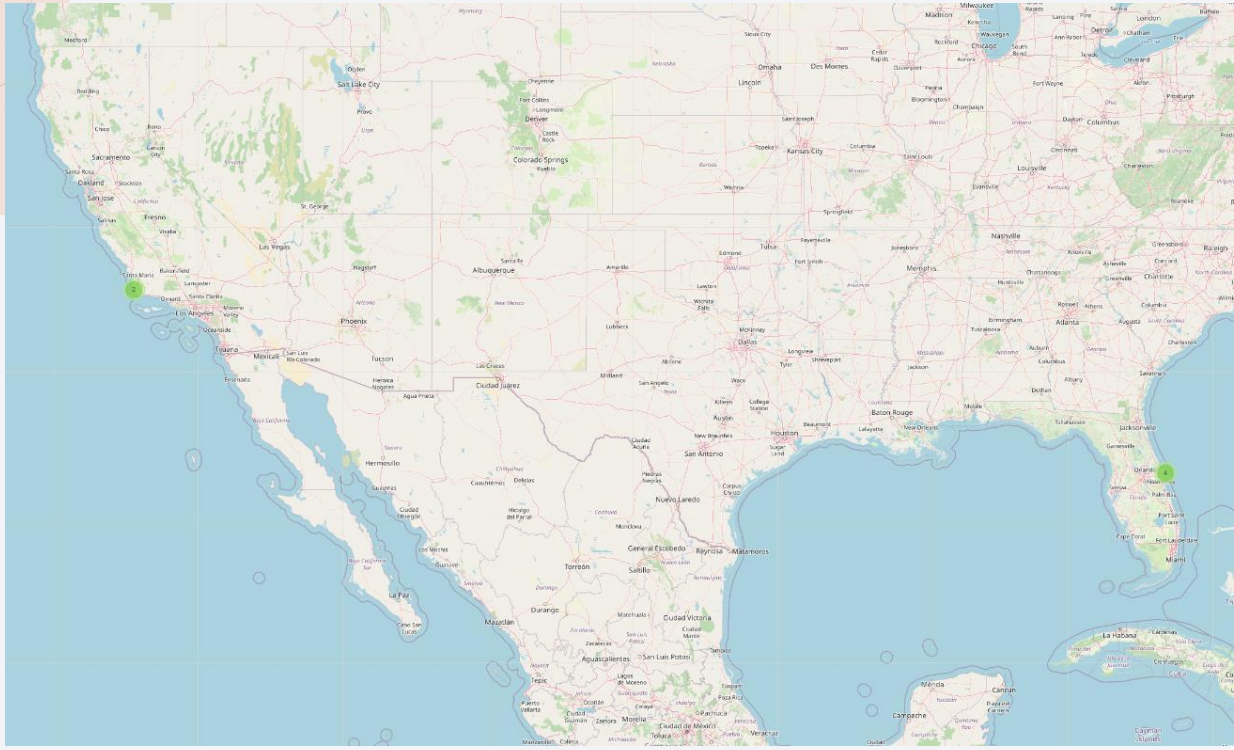
```
initial_location = [launch_sites_df['Latitude'].mean(), launch_sites_df['Longitude'].mean()]
spacex_map = folium.Map(location=initial_location, zoom_start=5)
for index, row in launch_sites_df.iterrows():
    marker=folium.Marker(
    location=[row['Latitude'], row['Longitude']],
    popup=row['LaunchSite'],
    radius=10,
    color='blue',
    fill=True,
    fill_color='blue'
)
    marker.add_to(spacex_map)
spacex_map
```

✓ 0.0s



- **Data Visualization:** You've successfully visualized geographic data on a map, which is a crucial step in understanding spatial relationships and patterns.
- **Marker Placement**: The blue markers indicate specific points of interest or data points, which can be used for further analysis or decision-making.
- **Code Integration:** The integration of code with the map interface suggests that you can dynamically update or manipulate the map based on your data inputs.

<Folium Map Screenshot 2>



Section 5

Predictive Analysis (Classification)

Classification Accuracy

```
print('Accuracy for Logistics Regression method:', logreg_cv.score(X_test, Y_test))
print( 'Accuracy for Support Vector Machine method:', svm_cv.score(X_test, Y_test))
print('Accuracy for Decision tree method:', tree_cv.score(X_test, Y_test))
print('Accuracy for K nearsdt neighbors method:', knn_cv.score(X_test, Y_test))
```

```
ccuracy for Logistics Regression method: 0.8333333333333334
ccuracy for Support Vector Machine method: 0.8333333333333334
ccuracy for Decision tree method: 0.6111111111111112
ccuracy for K nearsdt neighbors method: 0.8333333333333334
```

Based on the accuracy results of different machine learning classification methods:

- Logistic Regression and Support Vector Machine both achieved the highest accuracy of 83.33%.
- Decision Tree and K Nearest Neighbors both had lower accuracy scores of 61.11%.

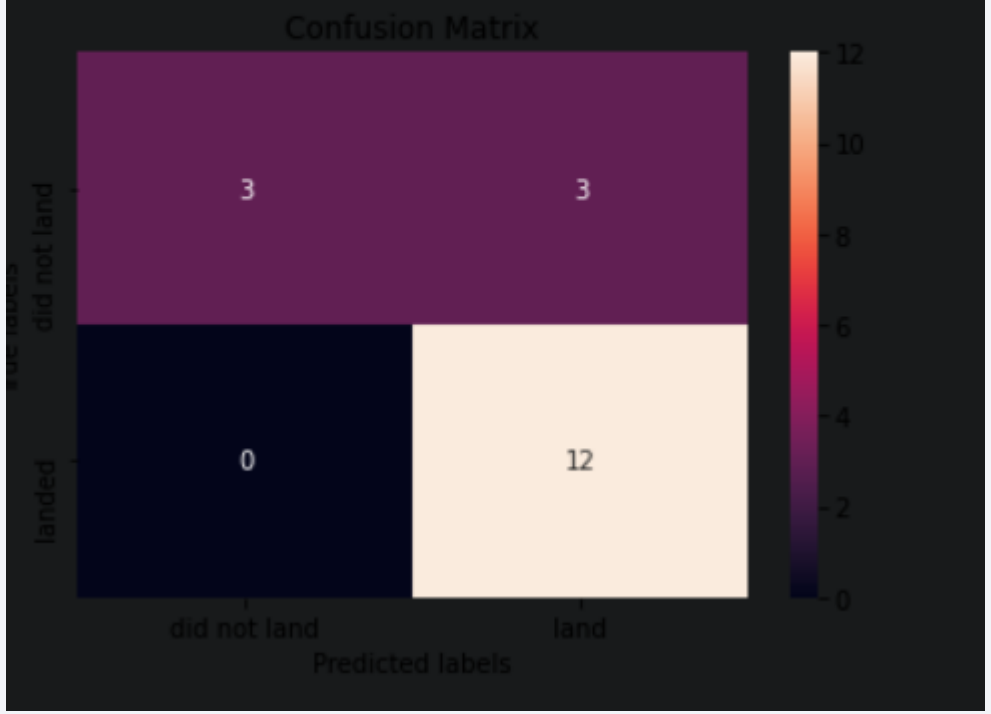
❖ **<u>Conclusion:</u>**

Logistic Regression and Support Vector Machine are the most effective models for your dataset, providing the highest accuracy.

Decision Tree and K Nearest Neighbors may require further tuning or might not be the best fit for this particular dataset.

Confusion Matrix

```
yhat=logreg_cv.predict(X_test)
plot_confusion_matrix(Y_test,yhat)
```



- **Conclusion:**
- **Model Accuracy**: The confusion matrix helps you understand the accuracy of your model by showing how many predictions were correct versus incorrect.
- **Error Analysis**: By examining the false positives and false negatives, you can identify areas where your model might need improvement.
- **Model Performance**: The balance between true positives and true negatives indicates how well your model is performing overall.

Executive Summary

- Summary of methodologies
- Summary of all results

Thank you!